

TRIE DATA STRUCTURE

Software Requirements Specification (SRS)

Trainee Implementation Guide

Version 2.0 | C++ Data Structures Training Module

Prepared for: Software Engineering Trainees

Language: C++ (Tree Structures, Recursion, Memory Management & String Processing)

INTERNAL TRAINING DOCUMENT — NOT FOR PUBLIC DISTRIBUTION

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional requirements for implementing a Trie (prefix tree) data structure in C++. It is intended to guide trainees through completing a partially-written codebase by specifying exactly what each method must do.

This version of the exercise extends the original insert / search / prefix / autocomplete implementation with word-count tracking, word removal, full memory cleanup, longest-prefix matching, an emptiness check, a full reset operation, and a limited-result variant of autocomplete. By the end of this exercise, trainees will demonstrate mastery of:

- Tree-based data structures and node design in C++
- Recursive tree traversal (DFS) for word enumeration and counting
- Prefix-based search and autocomplete algorithms, including result-limited variants
- Manual dynamic memory management: allocation, recursive deallocation, and leak-free deletion
- Recursive deletion of trie nodes that safely prunes now-unused branches
- Edge-case handling: empty strings, missing prefixes, single-character words, shared prefixes, and duplicate insertions

1.2 Project Overview

The Trie class stores a set of lowercase English words using a tree of TrieNode objects. Each node holds up to 26 child pointers (one per letter a–z) and a boolean flag that marks whether a complete word ends at that node. The Trie itself now also tracks a running wordCount so the total number of stored words can be reported in $O(1)$ time instead of being recomputed by a full traversal on every call.

The skeleton code — class declarations with empty method bodies and a full 13-part test suite in main() — is provided. Trainees must implement all method bodies marked // TODO, including four private helper methods and eight additional public methods beyond the original four core operations.

1.3 Scope

The Trie implementation covers the following operations:

- Insertion of words into the trie, with duplicate-safe word counting
- Exact-word search (full match required)
- Prefix existence check (partial match accepted)
- Autocomplete: retrieving all words that share a given prefix
- Autocomplete with a maximum result limit
- Removal of a single word, pruning any nodes left unused by the removal
- Counting the total number of stored words, and the number of words under a given prefix
- Retrieving every word currently stored in the trie
- Finding the longest prefix of an input word that exists as a path in the trie
- Checking whether the trie currently holds any words
- Clearing the trie back to its initial empty state
- Full and correct deallocation of every dynamically allocated node, both during clear() and on destruction

1.4 Definitions and Acronyms

Term	Definition
Trie	A tree data structure where each path from root to an end-of-word node spells out a stored word
TrieNode	A single node in the Trie holding 26 child pointers and an isEndOfWord flag
Prefix	A leading substring of a word (e.g. "app" is a prefix of "apple")
Autocomplete	Returning all complete words stored in the Trie that begin with a given prefix
DFS	Depth-First Search — a recursive traversal strategy used to enumerate or count words
isEndOfWord	A boolean flag on a TrieNode set to true when a complete inserted word terminates at that node
wordCount	A private int field on Trie that tracks the total number of unique words currently stored, maintained incrementally by insert() and remove()
Longest Prefix Match	The longest leading portion of an input string for which a path exists in the trie, whether or not that path ends on a complete word
Pruning	Deleting nodes that are left with no children and no isEndOfWord flag after a word is removed, so the trie does not retain unused memory
Trainee	A junior developer completing this exercise

2. Data Structure Design

2.1 TrieNode Class

Each node in the Trie has exactly two fields:

Field	Type	Description
children[26]	TrieNode*	Array of 26 pointers, one per lowercase letter a–z. children[0] = 'a', children[1] = 'b', ..., children[25] = 'z'. Initialize all to nullptr.
isEndOfWord	bool	Set to true when a complete word ends at this node. Initialize to false.

The TrieNode constructor is already provided and correctly initializes both fields. Do not modify it.

2.2 Trie Class

The Trie class contains two private fields:

Field	Type	Description
root	TrieNode*	The entry point of the trie. All operations begin by following children from this node.
wordCount	int	Running total of unique complete words currently stored. Incremented in insert() only when a word is newly marked as complete; decremented in remove() only when an existing word is actually deleted. countWords() simply returns this field.

2.3 Character-to-Index Mapping

Throughout the implementation you will need to convert a character to the corresponding array index. Use the formula:

```
int index = ch - 'a'; // 'a' -> 0, 'b' -> 1, ..., 'z' -> 25
```

The test suite inserts only lowercase words for most tests. Section 8.6 (Case Sensitivity) intentionally inserts mixed-case words like "Hello" and "WORLD" to verify that search() correctly returns NOT FOUND for lowercase lookups of those words. Do not add case-conversion logic — the exercise is deliberately case-sensitive. Note that `ch - 'a'` on an uppercase letter yields a negative value; the implementation is not required to guard against this, but you should understand why it makes uppercase entries behave inconsistently with normal a–z traversal (see Section 6.4 note in the getAllWords discussion).

3. Constructor and Destructor

3.1 Trie() — Default Constructor

Initializes the Trie with an empty root node and a zeroed word count so that all operations have a valid starting point.

- Allocate a new TrieNode and assign it to root
- Initialize wordCount to 0
- No words are stored yet — all children of root are nullptr and isEndOfWord is false

3.2 ~Trie() — Destructor

Frees every dynamically allocated TrieNode so the program does not leak memory when a Trie goes out of scope.

- Call the private helper deleteNodes(root)
- deleteNodes() must recursively free every descendant node before freeing the node passed to it (post-order), otherwise child pointers are freed after their parent, or leaked entirely

4. Core Public Methods

4.1 insert(string word)

Adds a word to the Trie by walking (and creating if necessary) a chain of nodes, one per character, and updates wordCount only if the word is new.

- Start at root
- For each character ch in word:
 - Compute index = ch - 'a'
 - If current->children[index] is nullptr, allocate a new TrieNode and assign it there
 - Advance current to current->children[index]
- After the loop, if current->isEndOfWord is currently false, increment wordCount (this word is newly complete)
- Set current->isEndOfWord = true on the final node

Example — inserting "cat":

```
root --[c]--> node_c --[a]--> node_a --[t]--> node_t (isEndOfWord = true)
```

Important: inserting the same word twice must not increment wordCount a second time — the count-then-increment check (only increment when isEndOfWord was previously false) is what keeps wordCount accurate under duplicate inserts.

4.2 search(string word)

Returns true if the exact word exists in the Trie (i.e., there is a complete path from root to an isEndOfWord node), false otherwise.

- Start at root
- For each character ch in word:
 - Compute index = ch - 'a'
 - If current->children[index] is nullptr: return false immediately
 - Advance current to current->children[index]
- After the loop, return current->isEndOfWord

Stored words	Search query	Expected result
apple, banana	apple	true — exact match
apple, banana	app	false — prefix only, isEndOfWord is false
apple, banana	pear	false — path doesn't exist
apple, banana	"" (empty)	false — root->isEndOfWord is false

4.3 startsWith(string prefix)

Returns true if any word currently stored in the Trie begins with the given prefix. The prefix itself does not have to be a complete stored word.

- Start at root
- For each character ch in prefix:
 - Compute index = ch - 'a'
 - If current->children[index] is nullptr: return false immediately

- Advance current to `current->children[index]`
- After the loop, return true (the path exists regardless of `isEndOfWord`)

Key difference from `search()`: `startsWith()` returns true as long as the path exists; it never checks `isEndOfWord`. An empty prefix should return true because every trie (even an empty one) can be said to have the empty-string path.

4.4 `autocomplete(string prefix)`

Returns a vector of all complete words stored in the Trie that start with the given prefix. If no such words exist, returns an empty vector.

- Navigate to the node at the end of the prefix path (same traversal as `startsWith()`):
 - If any character in the prefix has no child node, return the empty vector immediately
- Once at the end-of-prefix node, call the private helper `findAllWords()` passing that node, the prefix string as the initial `currentWord`, and the suggestions vector
- Return the populated suggestions vector

Edge case — empty prefix: when prefix is "", the end-of-prefix node is root itself. Calling `findAllWords(root, "", results)` will then collect every word in the trie, which is the expected behavior shown in Test 4.

4.5 `autocomplete(string prefix, int limit)` — Overload

Returns at most `limit` complete words starting with the given prefix. This overload reuses the same traversal and DFS logic as the single-argument version rather than duplicating it.

- Navigate to the end-of-prefix node exactly as in Section 4.4; if the prefix path does not exist, return an empty vector
- Call `findAllWords()` from that node to collect every matching word into a temporary vector (or directly into suggestions)
- If more results were collected than `limit`, keep only the first `limit` entries
- Return the (possibly truncated) suggestions vector

Because `findAllWords()` enumerates words in a deterministic a-to-z depth-first order (Section 6.1), truncating to the first `limit` entries after collection is sufficient and matches the expected output in Section 8.11 — it is not necessary to stop the recursion early, though doing so is a valid and slightly more efficient alternative.

5. Word Management Methods

5.1 remove(string word)

Deletes a complete word from the Trie, pruning any nodes that are left with no children and no other complete word passing through them, and decrements wordCount if — and only if — the word actually existed.

- Call the private helper removeHelper(root, word, 0)
- removeHelper() returns true when the child it was just called on can safely be deleted by its caller
- Only decrement wordCount if the word was actually found and removed — calling remove() on a word that was never inserted must leave wordCount unchanged

Example: Insert "apple", "app". Remove "apple". "app" should still exist, because the node for "app" has isEndOfWord = true and removeHelper() must never delete a node that still marks the end of another stored word.

5.2 countWords()

Returns the total number of unique complete words currently stored in the Trie.

- Simply return wordCount

This must run in $O(1)$ — do not re-traverse the trie here. wordCount is what makes that possible.

5.3 countWordsWithPrefix(string prefix)

Returns the number of complete words that begin with the given prefix.

- Navigate to the end-of-prefix node exactly as in startsWith(); if the path does not exist, return 0 immediately
- Otherwise, return countWordsFromNode(node), the private helper that counts isEndOfWord nodes in a subtree

5.4 getAllWords()

Returns every complete word currently stored in the Trie.

- Call findAllWords(root, "", words) and return the resulting vector

This is functionally equivalent to autocomplete("") and should reuse the same private helper rather than re-implementing the traversal.

5.5 longestPrefixOf(string word)

Returns the longest leading portion of word for which a path exists in the Trie. This is a path match, not a stored-word match — the returned prefix does not need to end on a node where isEndOfWord is true.

- Start at root with an empty built-up string
- For each character ch in word:
 - Compute index = ch - 'a'
 - If current->children[index] is nullptr, stop the loop immediately
 - Otherwise advance current to current->children[index] and append ch to the built-up string
- Return the built-up string once the loop ends or breaks

Example:

```
Trie contains:  
"apple"  
"application"
```



```
Input:  "appreciate"
```

```
Output: "app"
```

Walking "appreciate" matches nodes for 'a', 'p', 'p' (shared by both stored words), but the node for "app" has no child for 'r', so the walk stops there and "app" is returned — even though "app" alone was never inserted as a complete word.

5.6 isEmpty()

Returns true if the Trie currently holds no words, false otherwise.

- Return wordCount == 0

5.7 clear()

Removes every word from the Trie and returns it to the same state as a freshly constructed Trie.

- Call deleteNodes(root) to free the existing root and every descendant node
- Allocate a new TrieNode and assign it to root, exactly as in the constructor
- Reset wordCount to 0

After clear(), isEmpty() must return true and countWords() must return 0.

6. Private Helper Methods

6.1 findAllWords(TrieNode* node, string currentWord, vector<string>& results)

Recursively performs a depth-first traversal of the trie starting from node, building up currentWord character by character, and appending complete words to results.

Algorithm (implement these three steps exactly):

- Step 1 — Check for word completion: if node->isEndOfWord is true, push currentWord onto results
- Step 2 — Recurse into every existing child: for each index i from 0 to 25, if node->children[i] is not nullptr, call findAllWords(node->children[i], currentWord + char('a' + i), results)
- Step 3 — Base case (implicit): when a leaf node is reached and no children exist, all recursive calls return naturally

Example traversal for prefix "ap" with words {"apple", "appetizer"} stored:

```
findAllWords(node_p, "ap", results)
-> child 'p': findAllWords(node_pp, "app", results)
    -> child 'e': findAllWords(node_e, "appe", results)    [appetizer branch]
        -> ... eventually pushes "appetizer"
    -> child 'l': findAllWords(node_l, "appl", results)    [apple branch]
        -> child 'e': isEndOfWord=true -> push "apple"
```

Because Step 2 iterates children in index order (0 through 25, i.e. 'a' through 'z'), every method built on top of findAllWords() — autocomplete(), autocomplete() with a limit, and getAllWords() — returns words in deterministic alphabetical-by-branch order. This is why Section 8 can list exact expected output.

6.2 deleteNodes(TrieNode* node)

Recursively frees node and every node reachable from it. Used by both the destructor and clear().

- Base case: if node is nullptr, return immediately
- For each index i from 0 to 25, if node->children[i] is not nullptr, call deleteNodes(node->children[i])
- After all children have been freed, delete node

Freeing children before the parent (post-order) is required — deleting node first would make its children pointers unreachable, leaking every node beneath it.

6.3 countWordsFromNode(TrieNode* node)

Recursively counts how many complete words exist in the subtree rooted at node, including node itself if it marks a complete word.

- Initialize count = 0
- If node->isEndOfWord is true, count = 1
- For each index i from 0 to 25, if node->children[i] is not nullptr, add countWordsFromNode(node->children[i]) to count
- Return count

This helper backs countWordsWithPrefix() and is intentionally separate from findAllWords() — it only needs a running total, not the words themselves, and should not build or pass strings.

6.4 removeHelper(TrieNode* node, string word, int index)

Recursively walks the path for word and, on the way back up the call stack, deletes any node that is left with no children and no isEndOfWord flag as a result of the removal. Returns true to tell the caller "you may delete the child pointer that led you here".

Algorithm:

- Base case — reached the end of the word (`index == word.length()`):
 - If `node->isEndOfWord` is false, the word was never stored here — return false (nothing to delete, nothing was removed)
 - Otherwise set `node->isEndOfWord = false`
 - Return true if node now has no children (it can be safely deleted by the caller); return false if it still has at least one child (it must be kept)
- Recursive case — `index < word.length()`:
 - Compute `idx = word[index] - 'a'`
 - If `node->children[idx]` is `nullptr`, the word does not exist in the trie — return false
 - Recursively call `shouldDeleteChild = removeHelper(node->children[idx], word, index + 1)`
 - If `shouldDeleteChild` is true: delete `node->children[idx]`, then set `node->children[idx] = nullptr`
 - Return true only if, after that deletion, node itself now has no remaining children AND `node->isEndOfWord` is false; otherwise return false

This is the same "app" / "apple" example from Section 5.1: removing "apple" deletes the node for the final 'e', then unwinds back through 'l', 'p', 'p'. At the second 'p' node (the one spelling "app"), `isEndOfWord` is true, so `removeHelper()` returns false there and the "app" path — and therefore the word "app" — is preserved.

7. Edge Cases

Your implementation must correctly handle all of the following situations, which are explicitly tested in `main()`:

Input / Situation	Expected Behavior	Why It Matters
<code>search("")</code> — empty string	Returns false	<code>root->isEndOfWord</code> is initialized to false and no insert touches the root node directly
<code>startsWith("")</code> — empty prefix	Returns true	The empty-string path always exists from root
<code>autocomplete("")</code> — empty prefix	Returns all stored words	Traversal starts from root, finding every word in the trie
Prefix that exists but has no complete word below it	<code>autocomplete</code> returns empty vector	<code>findAllWords</code> finds no <code>isEndOfWord</code> nodes, so results stays empty
Prefix not present in the trie	<code>startsWith</code> and <code>autocomplete</code> return false / empty; <code>countWordsWithPrefix</code> returns 0	Path traversal hits nullptr before finishing the prefix
Mixed-case insert: "Hello", "WORLD"	<code>search("hello")</code> -> NOT FOUND; <code>search("Hello")</code> -> FOUND	The implementation is case-sensitive; 'H' - 'a' produces a different index than 'h' - 'a'
Word that is a prefix of another stored word	Both <code>search("app")</code> and <code>search("apple")</code> are handled independently by <code>isEndOfWord</code>	If "app" is inserted, its terminal node has <code>isEndOfWord=true</code> even though "apple" extends through it
Removing a word that is a prefix of another stored word	<code>remove("apple")</code> after inserting "app" and "apple" leaves "app" intact	<code>removeHelper()</code> must not delete or unmark a node whose <code>isEndOfWord</code> is still needed by a shorter stored word
Removing a word that was never inserted	<code>wordCount</code> is unchanged; trie structure is unchanged	<code>removeHelper()</code> returns false as soon as a required child pointer is missing or <code>isEndOfWord</code> is already false
Inserting the same word twice	<code>wordCount</code> increases by 1, not 2	<code>insert()</code> only increments <code>wordCount</code> when <code>isEndOfWord</code> transitions from false to true
<code>longestPrefixOf()</code> on a word sharing only a partial path	Returns the matched path even if it is not itself a complete stored word	e.g. "app" is returned for "appreciate" even though "app" was never inserted on its own
<code>clear()</code> on a non-empty trie	<code>countWords()</code> becomes 0 and <code>isEmpty()</code> becomes true afterward	<code>deleteNodes()</code> must free the entire previous tree before root is reassigned
<code>autocomplete(prefix, limit)</code> when fewer matches exist than limit	Returns all matches (no padding, no error)	Truncation only applies when the number of matches exceeds limit

8. Method Summary

Method Signature	Input / Output	Purpose
Trie()	No input; no output	Initialize root node and wordCount
~Trie()	No input; no output	Free all dynamically allocated nodes
insert(string word)	string -> void	Add word to trie; update wordCount
search(string word)	string -> bool	True if exact word exists
startsWith(string prefix)	string -> bool	True if any word has this prefix
autocomplete(string prefix)	string -> vector<string>	All words starting with prefix
autocomplete(string prefix, int limit)	string, int -> vector<string>	At most limit words starting with prefix
remove(string word)	string -> void	Delete a word; prune unused nodes; update wordCount
countWords()	No input -> int	Total number of stored words ($O(1)$)
countWordsWithPrefix(string prefix)	string -> int	Number of stored words starting with prefix
getAllWords()	No input -> vector<string>	Every word currently stored
longestPrefixOf(string word)	string -> string	Longest path-matching prefix of word
isEmpty()	No input -> bool	True if the trie holds no words
clear()	No input; no output	Remove all words and free all nodes
findAllWords(node, word, results) [private]	TrieNode*, string, vector<string>& -> void	DFS helper to collect all words from a subtree
deleteNodes(node) [private]	TrieNode* -> void	Recursively free a subtree
countWordsFromNode(node) [private]	TrieNode* -> int	Count complete words in a subtree
removeHelper(node, word, index) [private]	TrieNode*, string, int -> bool	Recursively remove a word and report if the caller's child can be pruned

9. Expected Test Output

The test suite in `main()` will run automatically when your implementation is complete. Compile and run with:

```
g++ -std=c++17 -Wall -Wextra trie.cpp -o trie && ./trie
```

9.1 Test 1 — Basic Insertion and Search

```
Inserted: apple
Inserted: banana
Inserted: orange
Inserted: grape
Inserted: kiwi
Search 'apple': FOUND
Search 'banana': FOUND
Search 'orange': FOUND
Search 'grape': FOUND
Search 'kiwi': FOUND
Search 'app': NOT FOUND (expected: NOT FOUND)
Search 'ban': NOT FOUND (expected: NOT FOUND)
Search 'ora': NOT FOUND (expected: NOT FOUND)
Search 'graph': NOT FOUND (expected: NOT FOUND)
Search 'kiwis': NOT FOUND (expected: NOT FOUND)
```

9.2 Test 2 — Prefix Checking

```
Prefix 'app': EXISTS
Prefix 'ban': EXISTS
Prefix 'ora': EXISTS
Prefix 'grap': EXISTS
Prefix 'k': EXISTS
Prefix 'x': DOESN'T EXIST (expected: DOESN'T EXIST)
Prefix 'yield': DOESN'T EXIST (expected: DOESN'T EXIST)
Prefix 'zed': DOESN'T EXIST (expected: DOESN'T EXIST)
Prefix 'micro': DOESN'T EXIST (expected: DOESN'T EXIST)
Prefix 'nano': DOESN'T EXIST (expected: DOESN'T EXIST)
```

9.3 Test 3 — Autocomplete

```
Autocomplete for 'a': apple
Autocomplete for 'b': banana
Autocomplete for 'o': orange
Autocomplete for 'g': grape
Autocomplete for 'k': kiwi
Autocomplete for 'ap': apple
Autocomplete for 'ban': banana
Autocomplete for 'ora': orange
```

```
Autocomplete for 'gr': grape
Autocomplete for 'ki': kiwi
```

9.4 Test 4 — Edge Cases

```
Search empty string: NOT FOUND (expected: NOT FOUND)
Starts with empty prefix: EXISTS (expected: EXISTS)
Autocomplete for empty string: apple, banana, grape, kiwi, orange (expected: all words)
```

9.5 Test 5 — Additional Words

```
Inserted: application
Inserted: appetizer
Inserted: banister
Inserted: bandana
Inserted: oracle
Inserted: grapefruit
Search 'application': FOUND
Search 'appetizer': FOUND
Search 'banister': FOUND
Search 'bandana': FOUND
Search 'oracle': FOUND
Search 'grapefruit': FOUND
Autocomplete for 'app': appetizer, apple, application
Autocomplete for 'ban': banana, bandana, banister
Autocomplete for 'ora': oracle, orange
Autocomplete for 'gra': grape, grapefruit
```

Note on ordering: `findAllWords()` visits children in index order 0–25 ('a' before 'z'), so wherever a prefix has multiple matches, the branch whose next letter is alphabetically earlier is listed first — even if that branch's word is longer overall (e.g. "appetizer" before "apple" because 'e' precedes 'l' as the next letter after "app").

9.6 Test 6 — Case Sensitivity

```
Search 'hello': NOT FOUND
Search 'Hello': FOUND
Search 'WORLD': FOUND
Search 'world': NOT FOUND
```

9.7 Test 7 — Word Counting

```
Total words: 13
Words starting with 'app': 3
Words starting with 'ban': 3
Words starting with 'xyz': 0
```

Total reflects all 13 words inserted so far: apple, banana, orange, grape, kiwi, application, appetizer, banister, bandana, oracle, grapefruit, Hello, WORLD.

9.8 Test 8 — getAllWords

```
All words in Trie:
```

```
- appetizer
- apple
- application
- banana
- bandana
- banister
- grape
- grapefruit
- kiwi
- oracle
- orange
```

Only 11 of the 13 stored words appear here. "Hello" and "WORLD" were inserted using `ch - 'a'` on uppercase letters, which does not land in the valid `children[0..25]` range used by a standard a–z traversal, so they are not reachable by the normal DFS that `findAllWords()` performs from root. This is expected — Section 6.1 and 6.4 traversal logic operates strictly over indices 0 through 25.

9.9 Test 9 — longestPrefixOf

```
Longest prefix of 'application': application
Longest prefix of 'appreciate': app
Longest prefix of 'banana': banana
Longest prefix of 'band': band
Longest prefix of 'grapefruit': grapefruit
Longest prefix of 'unknown':
```

"band" matches in full because "bandana" passes through a node spelling "band", even though "band" itself was never inserted as a complete word — `longestPrefixOf()` only requires the path to exist. "unknown" returns an empty string because no stored word begins with 'u'.

9.10 Test 10 — Remove

```
Before removing 'apple': FOUND
After removing 'apple': NOT FOUND
Checking 'application': FOUND
Checking 'appetizer': FOUND
```

9.11 Test 11 — Autocomplete with Limit

```
Maximum 2 suggestions for 'app': appetizer, application
```

Only two words now share the "app" prefix ("apple" was removed in Test 10), so the limit of 2 does not truncate anything — both remaining matches are returned.

9.12 Test 12 — isEmpty

```
Is Trie empty? NO
```

9.13 Test 13 — Clear

Words before clear: 12

Words after clear: 0

Is Trie empty after clear? YES

10. Common Mistakes & Tips

Mistake	Why It's Wrong	Correct Approach
Not setting <code>isEndOfWord = true</code> after insert loop	<code>search()</code> always returns false even for inserted words	After exiting the for loop in <code>insert()</code> , always set <code>current->isEndOfWord = true</code> on the last node
Returning true from <code>search()</code> when <code>isEndOfWord</code> is false	Prefixes like "app" are incorrectly reported as found words	<code>search()</code> must check and return <code>current->isEndOfWord</code> , not just whether the path exists
Confusing <code>search()</code> with <code>startsWith()</code> logic	Mixing them causes incorrect results for both functions	<code>search()</code> checks <code>isEndOfWord</code> at the end; <code>startsWith()</code> returns true as soon as the full prefix path is walked
Passing <code>currentWord</code> by value to <code>findAllWords</code> but not appending the character before recursing	Autocomplete returns only the prefix, never builds full words	Pass <code>currentWord + char('a' + i)</code> in each recursive call so each level appends one letter
Forgetting to handle <code>nullptr</code> in <code>findAllWords/deleteNodes/countWordsFromNode</code> loops	Program crashes with a null pointer dereference during traversal	Always guard: if (<code>node->children[i] != nullptr</code>) before recursing
Not navigating to the prefix-end node in <code>autocomplete()</code>	Returns all words in the trie regardless of prefix	Walk the prefix first (same loop as <code>startsWith()</code>), then call <code>findAllWords</code> from that node
Initializing <code>root</code> to <code>nullptr</code> in the constructor	First <code>insert()</code> call crashes immediately	Always allocate a <code>TrieNode</code> in the constructor: <code>root = new TrieNode()</code>
Incrementing <code>wordCount</code> on every <code>insert()</code> call unconditionally	Re-inserting the same word inflates the count	Only increment <code>wordCount</code> when <code>current->isEndOfWord</code> was false before the assignment
Deleting a node in <code>removeHelper()</code> while it still has other children	Removing one word corrupts or deletes sibling words that share the same path	Only return true (safe to delete) when the node has zero children AND <code>isEndOfWord</code> is false
Deleting a node in <code>removeHelper()</code> that is still marked <code>isEndOfWord</code> for a shorter word	Removing "apple" also deletes "app" if both were inserted	Check <code>node->isEndOfWord</code> in the same condition before allowing deletion
Freeing a node in <code>deleteNodes()</code> before recursing into its children	Child pointers become unreachable, leaking every node beneath the freed node	Recurse into all 26 children first, then delete the current node (post-order)
Requiring <code>isEndOfWord</code> in <code>longestPrefixOf()</code>	Returns "" instead of "app" for a word like "appreciate" when only "apple" and "application" are stored	Track the matched path length regardless of <code>isEndOfWord</code> — only stop when a child pointer is missing
Reassigning <code>root</code> without freeing the old tree in <code>clear()</code>	The previous tree leaks — its memory is never freed, just orphaned	Call <code>deleteNodes(root)</code> before allocating the new root

Mistake	Why It's Wrong	Correct Approach
Implementing autocomplete(prefix, limit) as an entirely separate traversal	Duplicates the DFS logic in Section 6.1 and risks inconsistent ordering	Reuse findAllWords() to collect matches, then truncate the result to limit

11. Implementation Checklist

Use this checklist to track your progress. Every item must be completed before submission.

11.1 Constructor, Destructor, and DFS Helper

#	Item	Done?
1	Trie() constructor — allocates root with new TrieNode() and sets wordCount = 0	☐
2	~Trie() destructor — calls deleteNodes(root)	☐
3	deleteNodes() — recurses into all 26 children before deleting the current node	☐
4	findAllWords() — if node->isEndOfWord, push currentWord to results	☐
5	findAllWords() — iterates all 26 children, recurses with currentWord + char('a' + i)	☐

11.2 Core Methods

#	Item	Done?
6	insert() — creates missing nodes along the word path	☐
7	insert() — increments wordCount only when isEndOfWord transitions from false to true	☐
8	insert() — sets isEndOfWord = true on the final node	☐
9	search() — returns false when a child pointer is nullptr mid-traversal	☐
10	search() — returns current->isEndOfWord (not just true) at end of traversal	☐
11	startsWith() — returns false when a child pointer is nullptr mid-traversal	☐
12	startsWith() — returns true after full prefix is walked (does not check isEndOfWord)	☐
13	autocomplete(prefix) — navigates to end-of-prefix node first	☐
14	autocomplete(prefix) — returns empty vector immediately if prefix path is missing	☐
15	autocomplete(prefix) — calls findAllWords from the end-of-prefix node with prefix as initial word	☐
16	autocomplete(prefix, limit) — reuses findAllWords() and truncates to limit results	☐

11.3 Word Management Methods

#	Item	Done?
17	countWordsFromNode() — counts isEndOfWord nodes across the full subtree	☐
18	countWords() — returns wordCount directly (O(1), no traversal)	☐
19	countWordsWithPrefix() — navigates to end-of-prefix node, returns 0 if missing	☐
20	countWordsWithPrefix() — otherwise returns countWordsFromNode() on that node	☐

#	Item	Done?
21	getAllWords() — calls findAllWords(root, "", words) and returns the result	☐
22	longestPrefixOf() — tracks matched path length without requiring isEndOfWord	☐
23	longestPrefixOf() — stops as soon as a required child pointer is nullptr	☐
24	isEmpty() — returns wordCount == 0	☐
25	clear() — frees the existing tree with deleteNodes() before reallocating root	☐
26	clear() — resets wordCount to 0	☐
27	removeHelper() — base case returns false if isEndOfWord is already false at the target depth	☐
28	removeHelper() — base case clears isEndOfWord and reports whether the node has no children	☐
29	removeHelper() — recursive case returns false immediately if a required child is missing	☐
30	removeHelper() — deletes and nullifies a child only when the recursive call reports it is safe	☐
31	removeHelper() — never reports a node deletable while it still has children or isEndOfWord = true	☐
32	remove() — calls removeHelper(root, word, 0) and decrements wordCount only if the word existed	☐

12. Grading Criteria

Criterion	Weight	Description
Correctness of Output	30%	All 13 test groups produce the exact expected console output shown in Section 9
Edge Case Handling	20%	Empty-string search/prefix/autocomplete, missing prefixes, mixed-case words, duplicate inserts, and prefix-is-also-a-word scenarios all handled correctly
Memory Management	20%	No leaks: deleteNodes() is used correctly by both the destructor and clear(); removeHelper() prunes exactly the nodes that are safe to prune, never more and never less
Recursive DFS / Removal Logic	15%	findAllWords(), countWordsFromNode(), and removeHelper() correctly traverse or unwind the tree without missing or over-deleting any branch
Code Quality	10%	Readable code, no redundant traversal loops, proper reuse of private helpers to avoid duplicating DFS or deletion logic across public methods
Compilation	5%	Code compiles without warnings using: g++ -std=c++17 -Wall -Wextra

Appendix — Quick Reference Card

A concise summary for quick lookup during implementation.

Method Signatures and Return Values

Method	Signature	Returns
Constructor	Trie()	—
Destructor	~Trie()	— (frees all nodes)
Insert word	void insert(string word)	Nothing
Search exact word	bool search(string word)	true if full word exists
Check prefix	bool startsWith(string prefix)	true if any word starts with prefix
Get suggestions	vector<string> autocomplete(string prefix)	All words sharing the prefix
Get limited suggestions	vector<string> autocomplete(string prefix, int limit)	At most limit words sharing the prefix
Remove word	void remove(string word)	Nothing (prunes unused nodes)
Count all words	int countWords()	Total unique words (O(1))
Count words with prefix	int countWordsWithPrefix(string prefix)	Words starting with prefix
Get every word	vector<string> getAllWords()	All stored words
Longest matching prefix	string longestPrefixOf(string word)	Longest path-matching prefix
Check emptiness	bool isEmpty()	true if no words are stored
Clear all words	void clear()	Nothing (resets to empty trie)
DFS helper (private)	void findAllWords(TrieNode* node, string currentWord, vector<string>& results)	Nothing (modifies results by reference)
Deletion helper (private)	void deleteNodes(TrieNode* node)	Nothing (frees the subtree)
Subtree counter (private)	int countWordsFromNode(TrieNode* node)	Word count in subtree
Removal helper (private)	bool removeHelper(TrieNode* node, string word, int index)	true if caller's child can be pruned

Traversal Logic at a Glance

Method	Walks a path?	Checks isEndOfWord?	Recurse subtree?
insert()	Yes (creates nodes)	Sets it at end	No

Method	Walks a path?	Checks isEndOfWord?	Recurse subtree?
search()	Yes (read-only)	Yes — returns it	No
startsWith()	Yes (read-only)	No	No
autocomplete(prefix)	Yes (read-only)	No (delegates to helper)	Via findAllWords()
autocomplete(prefix, limit)	Yes (read-only)	No (delegates to helper)	Via findAllWords(), then truncated
countWords()	No	No (reads wordCount)	No
countWordsWithPrefix()	Yes (read-only)	No (delegates to helper)	Via countWordsFromNode()
getAllWords()	No (starts at root)	No (delegates to helper)	Via findAllWords()
longestPrefixOf()	Yes (read-only)	No — path existence only	No
isEmpty()	No	No (reads wordCount)	No
clear()	No	No	Via deleteNodes(), then reallocates root
remove()	Yes (read + write)	Clears it at end	No (unwinds recursively instead)
findAllWords()	No (starts at given node)	Yes — appends word if true	Yes — all 26 children
deleteNodes()	No (starts at given node)	No	Yes — all 26 children, post-order
countWordsFromNode()	No (starts at given node)	Yes — counts if true	Yes — all 26 children
removeHelper()	Yes (follows word)	Reads and clears it	No (linear recursion along the word)

End of Document